

Purchasing Agent

Approach & Design — AI Purchasing Agent Assignment

A buyer support agent for a quick-commerce purchasing workflow. Given a purchasing situation — a system recommendation, a supplier shortfall, a demand signal that has moved, or a constraint that blocks the obvious action — it gathers the relevant operational data itself, decides what should happen, carries out the resulting action within its authority, and then checks that the action actually landed the way it expected. It does not assume the recommendation it is handed is correct — that is the starting premise of the whole design.

Why this shape

The goal is not a chatbot that answers questions about purchasing — it is a system that makes, executes, and validates purchasing decisions. So the core of this project is not a prompt, it is a decision loop:

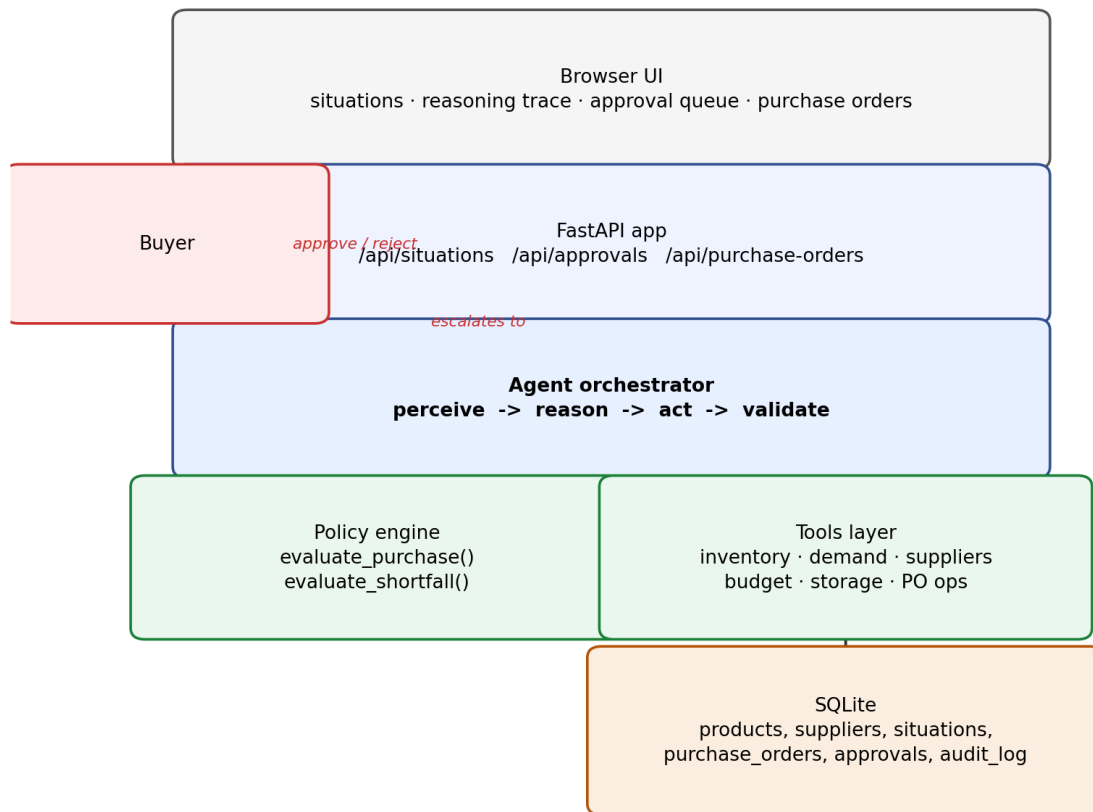
perceive → gather inventory, demand, suppliers, budget, storage via tools
reason → run the situation through a policy engine, get a decision and why
act → create or modify a purchase order, within an autonomy threshold
validate → re-check the constraint the decision relied on
escalate → hand anything uncertain, large, or failed to a human

Four scenarios map onto this loop as different entry points into the same two policies:

Scenario	Entry point	Policy used
1 — Recommendation review	Standing PO recommendation	evaluate_purchase
2 — Supplier can't fulfil	Supplier confirms a partial quantity	evaluate_shortfall
3 — Demand / forecast changed	No recommendation yet, actuals have moved	evaluate_purchase (demand re-based)
4 — Purchasing constraint	Recommendation exists, budget/storage won't allow it	evaluate_purchase (constraint-bound)

Reusing one policy for scenarios 1/3/4 is not a shortcut — a demand spike and a tight budget are both just different constraints on the same underlying question (how much should we buy, from whom, right now), so they belong in the same function rather than three near-duplicate copies of similar logic.

Architecture



Backend — Python, FastAPI, SQLite (stdlib sqlite3, no ORM). Seeded from `backend/seed_data.json` on every startup, so the demo is reproducible.

Frontend — a single static page (vanilla JS, no build step) served by the same FastAPI process. It shows the list of situations, lets you run the agent on one, and renders the full reasoning trace, the decision, the resulting purchase order, and the validation outcome, plus a buyer approval queue for anything the agent did not execute on its own.

Decision engine — deterministic and rule-based rather than a live model call. Every decision returns the list of factors that produced it, in plain language, and the same input always produces the same output. For a system that commits money and inventory, that auditability and reproducibility matters more than the flexibility of free-form reasoning, and it is directly testable without an API key or network access. The natural place a model would plug in is the "explain this for a specific buyer" step, or interpreting a free-text supplier email into the structured shortfall event this agent consumes — the tool boundary is designed so either could be added without touching the policy engine itself.

The feedback loop

This is what the brief asks about most directly, walked through with the actual example the seed data produces (situation SIT-1001):

- 1 The system recommends 800 units. The agent works out that, given current inventory, one order already in transit, and the fastest supplier's lead time, only about 338 units are actually needed to hold a safe cover window — already well under 800.
- 2 Storage is tighter still: only 300 units of capacity are available at the decision node. Storage becomes the binding constraint, so the agent places a PO for 300, not 800, and executes it immediately (300 units is within its auto-approval cost threshold).
- 3 At validation time the agent re-reads the storage figure. In the seed data, 40 units of that capacity were reserved by another workflow between the decision and the confirmation check — a stand-in for the everyday case of two systems reading a shared number a moment apart.
- 4 Validation catches that the PO it just placed (300) no longer fits (260 available), and — because 260 still clears the supplier's minimum order of 150 — trims the PO to 260 itself and logs why, rather than leaving an order in place the buyer never actually saw update. If trimming had gone below the supplier minimum, it escalates instead of silently failing.

The same validate-then-correct-or-escalate shape is what resolves Scenario 2 (a supplier shortfall is really "validation failed on an existing PO" one step earlier in the process) and what stops Scenario 4 from ever placing a non-compliant order in the first place.

Autonomy boundary — when a human gets involved

The agent acts on its own when a decision is accept/modify and: the resulting order cost is under \$5,000, demand looks stable (no unconfirmed spike), and a compliant quantity (at or above the supplier minimum) exists. Otherwise it stops short of acting and puts the proposal, with its full reasoning, in front of a buyer: a demand spike that has not yet sustained three days is flagged to investigate; a sustained spike is still escalated, on the principle that a step-change in demand deserves a human glance even when the resulting order is cheap; a situation where no quantity satisfies both the supplier minimum and budget/storage is escalated outright; and an alternate-supplier premium above 10%, or above the cost threshold, on a shortfall is proposed rather than auto-placed. The threshold and what counts as "stable" are constants at the top of the policy module and easy to argue with in either direction — that is a judgment call, not a hard rule from the brief.

How to run

```
cd backend
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
uvicorn app.main:app --reload
```

Open <http://localhost:8000>. The database is re-seeded from `seed_data.json` every time the server starts, so it is always in a known state. Click a situation, click Run agent, and read the trace. Anything that lands in the approval queue can be approved or rejected from the same page. No environment variables are required to run the demo.

Tests and evaluation

`test_scenarios.py` runs each seeded situation through the real agent (not a mock) and asserts on the decision, the resulting status, and — for SIT-1001 — the exact post-validation quantity, so the self-correction behaviour above is pinned down by a test, not just a demo. Two further cases are checked directly against the policy layer with synthetic inputs not worth seeding a whole UI situation for: a shortfall with no alternate supplier available, and a demand blip that has not sustained long enough to trust yet.

`evaluate.py` is the small evaluation scorecard the brief asks for, run as a standalone script rather than folded into pytest, so it reads like a report:

Situation	Decision	Match	Status	Match	Action	Validated	Escalated
SIT-1001	modify	yes	resolved_with_adjustment	yes	yes	yes	no
SIT-1002	accept	yes	resolved	yes	yes	yes	no
SIT-2001	sourced_alternate	yes	resolved	yes	yes	yes	no
SIT-3001	recommend_purchase	yes	pending_approval	yes	no	yes	yes
SIT-4001	escalate	yes	escalated	yes	no	yes	yes

For each situation this checks: was the decision what was expected, did it reach the expected status, did it take an action or correctly hold off, did validation pass (or correctly fail closed), and did it escalate when it was supposed to — mapping directly onto the brief's evaluation questions (was the decision correct, did it respect constraints, did it validate the result, what happens when the initial action does not work).

Data model

`seed_data.json` defines 5 products, 7 suppliers, 3 open purchase orders, and 5 situations — one per scenario, plus a second recommendation that is already well-calibrated as a contrast to the one that gets corrected. Everything is small enough to read end to end in one file.

Known limitations

- Single node, single buyer, no auth — this is a decision-and-execution demo, not a multi-tenant planning platform.
- Supplier confirmation, budget, and storage are all simulated via seed data rather than live integrations — the tools module is the seam where real systems would plug in.

- The storage drift that drives the validation-correction demo is seeded per situation rather than genuinely concurrent; the mechanism it stands in for (two reads of a shared counter racing) is real, the concurrency itself is not.
- No persistence across restarts by design — every run starts from the same seed so scenarios are reproducible on demand.